

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
Engenharia da Computação
Compiladores

Analizador Sintático

Alunos: Gabriel Bernalle Barbosa Matozinhos

Professora: Kecia Marques

Belo Horizonte, junho de 2025

Correção do Analisador Léxico

Começo o relatório informando que, ao analisar os resultados das saídas do Analisador Léxico para ter uma noção de onde parei e, posteriormente, continuar com a execução da tarefa, notei que o mesmo gerava uma definição errada de tokens para números, conforme mostrado na figura 1.

```
("10", ID)
(";", SEMICOLON)
("result", ID)
("=", ASSIGN)
("(", OP_ROUNDBRACK)
("a", ID)
("*", MUL)
("ch", ID)
(")", CL_ROUNDBRACK)
("/", DIV)
("(", OP_ROUNDBRACK)
("b", ID)
("5", ID)
("-", SUB)
("345.27", ID)
```

Figura 1 - Erro de tipos no Analisador Léxico. (IDE, Visual Studio Code)

Explicação da correção: Ao criar o fluxo de valores inteiros e flutuantes, foi definido o Token Type, porém, a saída foi direcionada para padrão da tabela de símbolos, que é a responsável por criar ID's. Para correção, bastou direcionar para a saída correta, que é a de "case 18", que serve para tokens já definidos. Imagem da correção na Figura 2.

```
Windows PowerShell
("10", INTEGER_CONST)
(";", SEMICOLON)
("result", ID)
("=", ASSIGN)
("(", OP_ROUNDBRACK)
("a", ID)
("*", MUL)
("ch", ID)
(")", CL_ROUNDBRACK)
("/", DIV)
("(", OP_ROUNDBRACK)
("b", ID)
("5", INTEGER_CONST)
("-", SUB)
("345.27", FLOAT_CONST)
```

Figura 2 - Analisador Léxico corrigido. (Terminal, Windows PowerShell)

Desenvolvimento do Analisador Sintático

O primeiro passo para o desenvolvimento do Analisador Sintático, foi comentar as chamadas do Analisador Léxico da Main, visto que uma restrição para essa etapa, é justamente não mostrar os testes do mesmo funcionando.

Após, foi criado o arquivo do Analisador Sintático que terá como principais funções:

- advance () -> Utilizado para buscar o próximo Token;
- eat() -> Utilizado para capturar todo o símbolo terminal;
- showError() -> Utilizado para mostrar, caso exista, o erro gerado;
- start() -> Função principal, que vai chamar o primeiro símbolo da gramática.

Como dito, quando chamado o método start(), o mesmo irá chamar o primeiro símbolo da gramática (program) e os métodos abaixo são implementadas chamando, executando e “comendo” o token, de acordo com a gramática definida no escopo do trabalho - Figura 3.

program	::= program [decl-list] begin stmt-list end
decl-list	::= decl {"," decl}
decl	::= type ":" ident-list ":"
ident-list	::= identifier {"," identifier}
type	::= int float char
stmt-list	::= stmt {"," stmt}
stmt	::= assign-stmt if-stmt while-stmt repeat-stmt read-stmt write-stmt
assign-stmt	::= identifier "=" simple_expr
if-stmt	::= if condition then [decl-list] stmt-list end if condition then [decl-list] stmt-list else declaration stmt-list end
condition	::= expression
repeat-stmt	::= repeat [decl-list] stmt-list stmt-suffix
stmt-suffix	::= until condition
while-stmt	::= stmt-prefix [decl-list] stmt-list end
stmt-prefix	::= while condition do
read-stmt	::= in "(" identifier ")"
write-stmt	::= out "(" writable ")"
writable	::= simple-expr literal
expression	::= simple-expr simple-expr relop simple-expr
simple-expr	::= term simple-expr addop term
term	::= factor-a term mulop factor-a
fator-a	::= factor ! factor "-" factor
factor	::= identifier constant "(" expression ")"
relop	::= "==" ">" ">=" "<" "<=" "!="
addop	::= "+" "-"
mulop	::= "*" "/" &&
constant	::= integer_const float_const char_const

Figura 3 - Gramática definida pelo escopo do trabalho (Roteiro Trabalho 2025.1)

Antes de passar para a fase da apresentação dos testes, vale a pena informar que foi mudado algumas coisas da gramática que eram erros, e que logo, mesmo o código estando correto do ponto de vista da gramática, poderia afetar os pontos, dado uma possível confusão.

A regra da gramática correta para decl-list e decl deveria ser:

```
decl-list ::= decl {";" decl}  
decl ::= type ":" ident-list
```

Porém, no documento contém um ; no fim de ident-list, o que ocorreria uma duplicação de ';' para a gramática funcionar.

```
decl-list      ::= decl {";" decl}  
decl           ::= type ":" ident-list ";"
```

Erro 1 - decl-list e decl (gramática do Roteiro Trabalho 2025.1)

Outro ponto, a gramática para simple-expr e term está de forma recursiva para esquerda; o método pode entrar em procSimpleExpr(), que chama procSimpleExpr() novamente, e assim por diante... Como não consome nenhum token antes da recursão, isso leva a recursão infinita (stack overflow) ou travamento.

```
simple-expr      ::= term | simple-expr addop term  
term            ::= factor-a | term mulop factor-a
```

Erro 2 - simple-exp e term (gramática do Roteiro Trabalho 2025.1)

Dito isso, mudei para essa forma:

```
simple-expr ::= term { addop term }  
term       ::= factor-a { mulop factor-a }
```

Por último, a regra procExpression(); a mesma exige um relop mesmo quando não precisa, com isso se não houver relop, ele dá erro mesmo que a simple-expr sozinha seja válida (como no if (media >= 15)).

```
expression      ::= simple-expr | simple-expr relop simple-expr
```

Erro 3 - expression (gramática do Roteiro Trabalho 2025.1)

Assim, ficou praticamente da mesma forma, apenas alterando a parte final para opcional:

```
expression ::= simple-expr [relop simple-expr]
```

Apresentação dos Testes:

Para execução dos casos, basta digitar no terminal:

```
javac .\Main.java | java Main '.\examples\[testeX].txt'
```

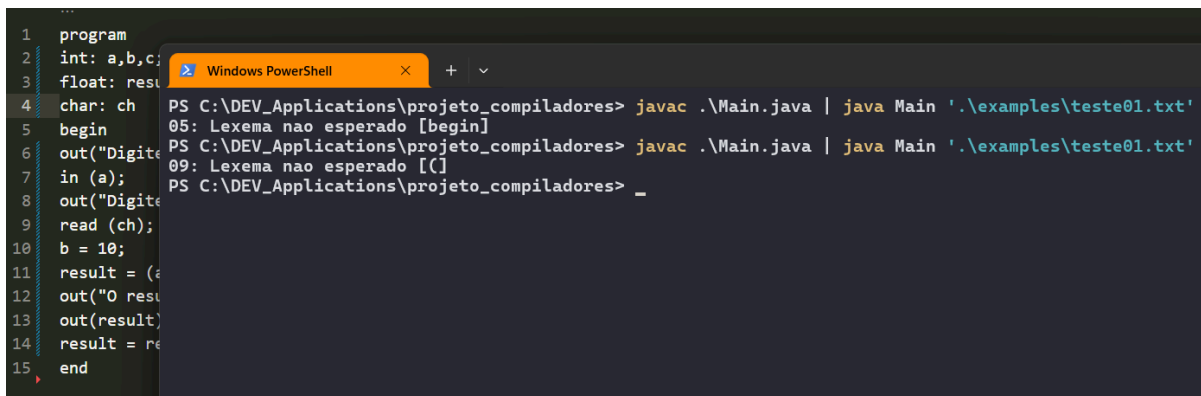
Substituindo [testeX] pelo teste que deseja analisar

Teste 01:

```
PS C:\DEV_Applications\projeto_compiladores> javac .\Main.java | java Main '.\examples\teste01.txt'
05: Lexema nao esperado [begin]
PS C:\DEV_Applications\projeto_compiladores> _
```

Caso Teste - 01 (Testes, Roteiro Trabalho 2025.1)

Erro ocorre pois a gramática não aceita um “,” após a última variável receber a atribuição, no caso o “char: ch;”. Removendo é possível notar que o erro será outro

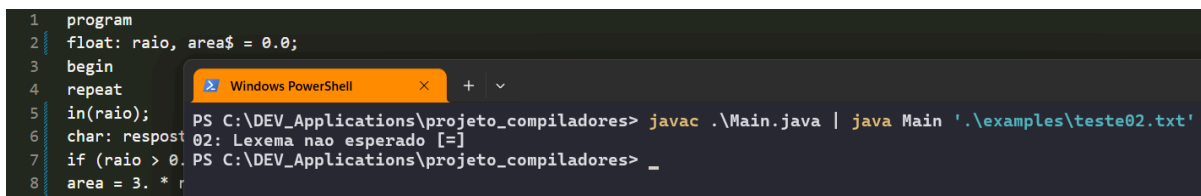


```
1 program
2 int: a,b,c;
3 float: resu
4 char: ch;
5 begin
6 out("Digite
7 in (a);
8 out("Digite
9 read (ch);
10 b = 10;
11 result = (a
12 out("O resu
13 out(result)
14 result = re
15 end
```

```
PS C:\DEV_Applications\projeto_compiladores> javac .\Main.java | java Main '.\examples\teste01.txt'
05: Lexema nao esperado [begin]
PS C:\DEV_Applications\projeto_compiladores> javac .\Main.java | java Main '.\examples\teste01.txt'
09: Lexema nao esperado [(]
PS C:\DEV_Applications\projeto_compiladores> _
```

Erro após remoção do “,” da última atribuição (ch)

Teste 02:



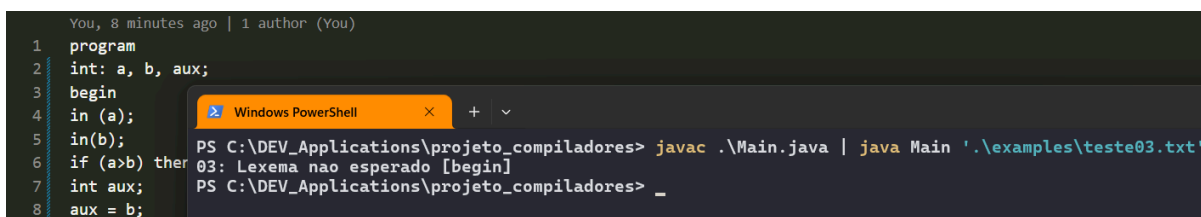
```
1 program
2 float: raio, area$ = 0.0;
3 begin
4 repeat
5 in(raio);
6 char: respost
7 if (raio > 0.
8 area = 3. * r
```

```
PS C:\DEV_Applications\projeto_compiladores> javac .\Main.java | java Main '.\examples\teste02.txt'
02: Lexema nao esperado [=]
PS C:\DEV_Applications\projeto_compiladores> _
```

Caso Teste - 02 (Testes, Roteiro Trabalho 2025.1)

Erro ocorre pois a gramática não permite atribuição logo quando um ID é definido, no caso, area\$ = 0.0;

Teste 03:

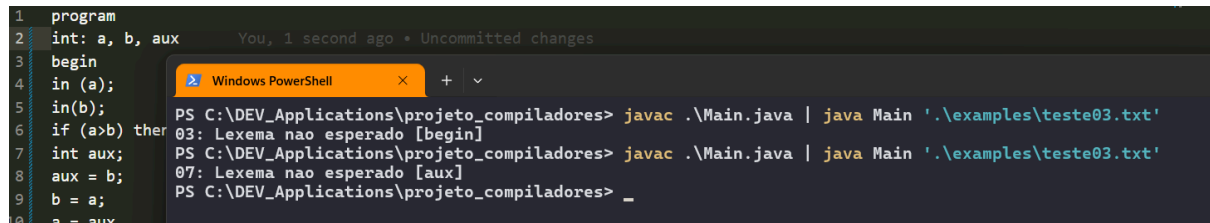


```
1 program
2 int: a, b, aux;
3 begin
4 in (a);
5 in(b);
6 if (a>b) then
7 int aux;
8 aux = b;
```

```
PS C:\DEV_Applications\projeto_compiladores> javac .\Main.java | java Main '.\examples\teste03.txt'
03: Lexema nao esperado [begin]
PS C:\DEV_Applications\projeto_compiladores> _
```

Caso Teste - 03 (Testes, Roteiro Trabalho 2025.1)

Mesmo erro do Teste 01. Corrigindo, nota-se que o erro será diferente.



```
1 program
2 int: a, b, aux
3 begin
4 in (a);
5 in(b);
6 if (a>b) then
7   int aux;
8   aux = b;
9   b = a;
10  a = aux;
```

```
PS C:\DEV_Applications\projeto_compiladores> javac .\Main.java | java Main '.\examples\teste03.txt'
03: Lexema nao esperado [begin]
PS C:\DEV_Applications\projeto_compiladores> javac .\Main.java | java Main '.\examples\teste03.txt'
07: Lexema nao esperado [aux]
PS C:\DEV_Applications\projeto_compiladores> _
```

Erro após remoção do “,” da última atribuição (aux)

Teste 04:



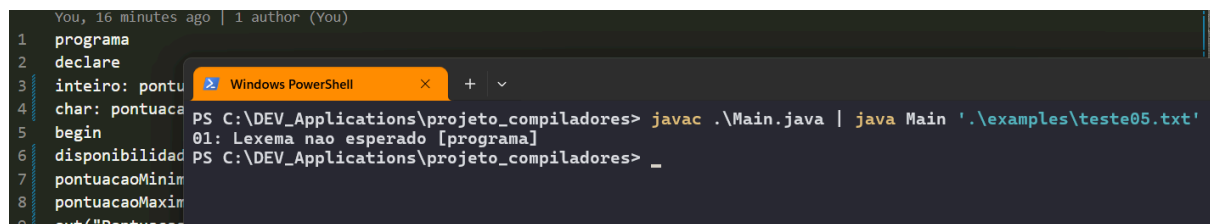
```
1 program
2 a, b, c, maior, outro: int;
3 begin
4 repeat
5   out("A");
6   in(a);
7   out("B");
8   in(b);
```

```
PS C:\DEV_Applications\projeto_compiladores> javac .\Main.java | java Main '.\examples\teste04.txt'
02: Lexema nao esperado [a]
PS C:\DEV_Applications\projeto_compiladores> _
```

Caso Teste - 04 (Testes, Roteiro Trabalho 2025.1)

Erro é causado pois após program é preciso ter um tipo, e no caso teste o que entra é um ID.

Teste 05:



```
1 programa
2 declare
3 inteiro: pontu
4 char: pontuaca
5 begin
6 disponibilid
7 pontuacaoMinim
8 pontuacaoMaxim
9 out("Pontuacac
```

```
PS C:\DEV_Applications\projeto_compiladores> javac .\Main.java | java Main '.\examples\teste05.txt'
01: Lexema nao esperado [programa]
PS C:\DEV_Applications\projeto_compiladores> _
```

Caso Teste - 05 (Testes, Roteiro Trabalho 2025.1)

Erro ocorre pois caso teste está escrito errado.

Teste 06:

Antes de mostrar que deu certo, tenho que avisar que modifiquei o caso teste que enviei anteriormente no Analisador Léxico, visto que o mesmo apresentou erro quando executado no Analisador Sintático.

```

examples > teste06.txt
You, 22 minutes ago | 1 author (You)
1  program
2  int : x, y, z;
3  float : media;
4  char : letra
5
6  begin
7      x = 10;
8      y = 20;
9      z = x + y;
10     media = (x + y + z) / 3.0;
11
12     if media >= 15 then
13         out("Aprovado")
14     else
15         out("Reprovado")
16     end;
17
18     repeat
19         in(x);
20         x = x - 1
21     until x <= 0
22
23 end

```

Novo Caso Teste 06 (IDE, Visual Studio Code)

Dito isso, abaixo a informação da correta execução do mesmo para o Analisador Sintático desenvolvido.

```

examples > teste06.txt
You, 23 minutes ago | 1 author (You)
1  program
2  int : x, y, z;
3  float : media;
4  char : letra
5
6  begin
7      x = 10;

```

Windows PowerShell

```

PS C:\DEV_Applications\projeto_compiladores> javac .\Main.java | java Main '.\examples\teste06.txt'
PS C:\DEV_Applications\projeto_compiladores> _

```

Caso Teste - 06 (Corrigido)